

# Gradient Descent for Linear Regression

Daffodil International University

June 2026

# Learning Objectives

- Review Linear Regression
- Understand Optimization
- Learn Mean Squared Error (MSE)
- Understand Gradient Descent intuitively and mathematically
- Perform manual parameter updates
- Implement Gradient Descent in Python
- Compare with Scikit-Learn

# Linear Regression Review

$$\hat{y} = wx + b$$

Where:

- $w$  = weight (slope)
- $b$  = bias (intercept)
- $x$  = input feature
- $\hat{y}$  = prediction

Goal: Find values of  $w$  and  $b$  that best fit the data.

# Why Do We Need Optimization?

Suppose:

- $w = 0, b = 0$
- Predictions are poor
- Infinite combinations of  $w$  and  $b$  are possible

Question:

How can we automatically find the best parameters?

Answer: Optimization.

# Cost Function (MSE)

For a dataset with  $n$  samples:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Properties:

- Always non-negative
- Lower value means better model
- Goal: Minimize  $J(w, b)$

# What is Gradient Descent?

Gradient Descent is an optimization algorithm that minimizes the cost function.

Basic Idea:

- 1 Calculate current error
- 2 Calculate gradient
- 3 Update parameters
- 4 Repeat until convergence

# Hill-Climbing Analogy

Imagine standing on a hill in dense fog.

- Current position = Current parameters
- Height = Cost value
- Slope = Gradient
- Walking downhill = Gradient Descent
- Lowest point = Global minimum

# Mathematical Derivation of MSE

Prediction:

$$\hat{y}_i = wx_i + b$$

Cost:

$$J(w, b) = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Substitute prediction equation:

$$J(w, b) = \frac{1}{n} \sum (y_i - (wx_i + b))^2$$



# Derivative Derivation: Step 1

Recall the prediction equation:

$$\hat{y}_i = wx_i + b$$

The Mean Squared Error (MSE) cost function is:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Substituting  $\hat{y}_i = wx_i + b$ :

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

Our goal is to find:

$$\frac{\partial J}{\partial w} \quad \text{and} \quad \frac{\partial J}{\partial b}$$

## Derivative Derivation: Step 2

Consider only one training example:

$$J = (y_i - (wx_i + b))^2$$

Notice that this expression contains:

- An outer function:  $(\cdot)^2$
- An inner function:  $y_i - (wx_i + b)$

To simplify, define:

$$u = y_i - (wx_i + b)$$

Then the cost becomes:

$$J = u^2$$

Now we can apply the Chain Rule.

# Derivative Derivation: Step 3

Chain Rule states:

$$\frac{dJ}{dw} = \frac{dJ}{du} \times \frac{du}{dw}$$

Since:

$$J = u^2$$

Using the Power Rule:

$$\frac{dJ}{du} = 2u$$

This is the derivative of the outer function.

# Derivative Derivation: Step 4

Recall:

$$u = y_i - (wx_i + b)$$

Expanding:

$$u = y_i - wx_i - b$$

Differentiate with respect to  $w$ :

$$\frac{du}{dw} = \frac{d}{dw}(y_i - wx_i - b)$$

Since:

$$\frac{d(y_i)}{dw} = 0$$

$$\frac{d(-wx_i)}{dw} = -x_i$$

$$\frac{d(-b)}{dw} = 0$$

Therefore:

$$\frac{du}{dw} = -x_i$$

## Derivative Derivation: Step 5

Combining the results using the Chain Rule:

$$\begin{aligned}\frac{dJ}{dw} &= \frac{dJ}{du} \times \frac{du}{dw} \\ &= (2u)(-x_i) \\ &= -2x_i u\end{aligned}$$

Substituting back

$$u = y_i - \hat{y}_i$$

gives

$$\frac{dJ}{dw} = -2x_i(y_i - \hat{y}_i)$$

This is the gradient for one training sample.

# Derivative Derivation: Step 6

For all training samples:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Therefore,

$$\frac{\partial J}{\partial w} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - \hat{y}_i)$$

This is the final gradient equation for the weight.

## Derivative Derivation: Step 7

Now derive the gradient with respect to the bias.

Using Chain Rule:

$$\frac{dJ}{db} = \frac{dJ}{du} \times \frac{du}{db}$$

We already know:

$$\frac{dJ}{du} = 2u$$

and

$$u = y_i - wx_i - b$$

Differentiating with respect to  $b$ :

$$\frac{du}{db} = -1$$



# Derivative Derivation: Step 8

Combining the results:

$$\begin{aligned}\frac{dJ}{db} &= (2u)(-1) \\ &= -2u\end{aligned}$$

Substituting back:

$$u = y_i - \hat{y}_i$$

gives

$$\frac{dJ}{db} = -2(y_i - \hat{y}_i)$$

For all training samples:

$$\frac{\partial J}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

# Key Takeaways

## Weight Gradient

$$\frac{\partial J}{\partial w} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - \hat{y}_i)$$

## Bias Gradient

$$\frac{\partial J}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

# Gradient Descent Update Rule

$$w_{new} = w - \alpha \frac{\partial J}{\partial w}$$

$$b_{new} = b - \alpha \frac{\partial J}{\partial b}$$

Where  $\alpha$  is the learning rate.

# Worked Example Dataset

| x | y |
|---|---|
| 1 | 2 |
| 2 | 4 |
| 3 | 6 |

Initialize:

$$w = 0, \quad b = 0, \quad \alpha = 0.1$$

# Initial Predictions

$$\hat{y} = wx + b$$

Since  $w = 0$  and  $b = 0$ :

| x | Actual | Predicted |
|---|--------|-----------|
| 1 | 2      | 0         |
| 2 | 4      | 0         |
| 3 | 6      | 0         |

$$J = \frac{(2 - 0)^2 + (4 - 0)^2 + (6 - 0)^2}{3}$$
$$= \frac{56}{3} = 18.667$$

Large cost indicates poor predictions.

# Gradient Calculation for Weight

$$\begin{aligned}\frac{\partial J}{\partial w} &= -\frac{2}{3}[(1)(2) + (2)(4) + (3)(6)] \\ &= -18.667\end{aligned}$$



# Gradient Calculation for Bias

$$\begin{aligned}\frac{\partial J}{\partial b} &= -\frac{2}{3}(2 + 4 + 6) \\ &= -8\end{aligned}$$

# Parameter Update (Iteration 1)

$$w = 0 - 0.1(-18.667) = 1.867$$

$$b = 0 - 0.1(-8) = 0.8$$

New model:

$$\hat{y} = 1.867x + 0.8$$

# Predictions After Iteration 1

| x | Actual | Prediction |
|---|--------|------------|
| 1 | 2      | 2.667      |
| 2 | 4      | 4.534      |
| 3 | 6      | 6.401      |

Cost drops dramatically.

## Iteration 2: Error Calculation

Errors:

| x | Actual | Prediction | Error  |
|---|--------|------------|--------|
| 1 | 2      | 2.667      | -0.667 |
| 2 | 4      | 4.534      | -0.534 |
| 3 | 6      | 6.401      | -0.401 |

## Iteration 2: Gradients

$$\frac{\partial J}{\partial w} = -\frac{2}{3}[(1)(-0.667) + (2)(-0.534) + (3)(-0.401)] \approx 1.959$$

$$\frac{\partial J}{\partial b} \approx 1.068$$

## Iteration 2: Parameter Update

$$w_2 = 1.867 - 0.1(1.959) = 1.671$$

$$b_2 = 0.8 - 0.1(1.068) = 0.693$$

New model:

$$\hat{y} = 1.671x + 0.693$$

# Multiple Iterations Table

| Iteration | Weight | Bias  | Cost   |
|-----------|--------|-------|--------|
| 0         | 0.000  | 0.000 | 18.667 |
| 1         | 1.867  | 0.800 | 0.297  |
| 2         | 1.671  | 0.693 | 0.074  |

# Learning Rate Effect

- Small Learning Rate: Slow convergence
- Good Learning Rate: Stable convergence
- Large Learning Rate: Overshooting and divergence

Typical values:

0.001, 0.01, 0.1



# Convergence Visualization

As iterations increase:

- Cost decreases
- Parameters stabilize
- Model converges

Goal:

$$\nabla J \approx 0$$

# Advantages and Limitations

## Advantages:

- Simple
- Scalable
- Used in Deep Learning

## Limitations:

- Learning-rate sensitive
- May converge slowly
- Requires iterative updates

# Summary

- Linear Regression predicts using  $\hat{y} = wx + b$
- MSE measures error
- Gradients determine update direction
- Gradient Descent minimizes the cost
- Repeated iterations improve the model

# In-Class Exercise

Dataset:

| x | y |
|---|---|
| 1 | 3 |
| 2 | 5 |
| 3 | 7 |

Tasks:

- 1 Initialize  $w = 0, b = 0$
- 2 Calculate MSE
- 3 Compute gradients
- 4 Perform one update

Thank You

Questions?